NFA Studio

Natural-Language-to-Automaton Workbench

Product & Engineering Specification · Version 1.0

Document property	Value
Purpose	Build a tool where a user types any supported language description and receives a verified NFA, its transition table, diagram and a step-by-step simulator.
Differentiator	Every automaton ships with a machine-checked correctness badge: it is compared against an independently built reference, not against itself.
Audience	Developers, reviewers and students implementing or evaluating the tool.
Scope	Regular languages expressible in the Language Specification IR (LSIR) defined in Section 5.
Status	Ready for implementation. Section 15 gives phased exit criteria.

One-sentence design: the user's words become a typed specification (LSIR); a deterministic compiler turns it into one canonical ε-NFA; an independent oracle and reference DFA prove the result; the table, diagram and simulator are pure views of that single model.

Contents

Contents 2

1. Vision and scope 4

1.1 What makes this design distinct . . . 4

1.2 Goals and non-goals . . . 4

2. Design principles 4

3. System architecture 5

3.1 Recommended stack . . . 5

3.2 Module layout . . . 6

4. Formal foundations used 6

5. Language Specification IR (LSIR) 6

5.1 Node catalogue . . . 7

5.2 Document shape . . . 7

5.3 Normalisation and limits . . . 7

6. Interpretation and clarification 7

6.1 Two-tier interpreter . . . 7

6.2 Ambiguity taxonomy . . . 8

6.3 Clarification state machine . . . 8

6.4 Non-regular detection . . . 8

7. Construction library 9

7.1 Atom constructions . . . 9

7.2 Combinators . . . 9

7.3 State-explosion policy . . . 9

7.4 Deterministic naming . . . 10

7.5 Worked construction: strings ending with ab over {a, b} . . . 10

7.6 Worked construction: not containing aa over {a, b} . . . 10

8. Verification ladder 10

8.1 Two independent references . . . 11

8.2 Levels . . . 11

8.3 Counterexample search . . . 11

9. Simulation engine 11

9.1 Algorithm . . . 11

9.2 Trace record . . . 12

9.3 Behaviour rules . . . 12

9.4 Interactive behaviour . . . 12

10. Table and diagram generation 12

10.1 Transition table . . . 12

10.2 State diagram . . . 12

10.3 Consistency check . . . 12
11. API contract (v1) 13
11.1 Examples . . . 13
11.2 Error codes . . . 14
12. User experience 14
12.1 Main flow . . . 14
12.2 Quality requirements . . . 14
13. Security and model governance 15
14. Testing and acceptance 15
14.1 Test layers . . . 15
14.2 Benchmark of unseen prompts (benchmark_v1) . . . 15
14.3 Acceptance criteria . . . 16
15. Implementation roadmap 16
16. Risks and open decisions 17
Appendix A. Glossary 17
Appendix B. Pseudocode: ε-closure and reconstruction 17

1. Vision and scope

NFA Studio lets a student describe a language in plain English (for example “binary strings with an even number of 1s that contain 00”) and immediately see the formal automaton: five-tuple, transition table, state diagram and an animated simulation. The tool does not trust its own interpretation or construction blindly. It proves each result with a second, independent computation and reports exactly how strong that proof is.

1.1 What makes this design distinct

- **Verification ladder (V0–V3).** Each automaton is graded: structural check, bounded exhaustive check against an oracle, exact equivalence against a derivative-based reference DFA, and property tests. The UI shows the highest level reached.
- **Two independent compilers.** The production compiler builds an ε-NFA by composition. A separate reference compiler uses Brzozowski derivatives to build a DFA. Agreement between them is the correctness evidence.
- **Shortest counterexample.** When the two disagree, the system returns the shortest string on which they differ, found by breadth-first search on the product automaton.
- **State meanings.** Every state carries a plain-language meaning (“seen a, waiting for b”) shown on hover and in the explanation.
- **Witness path.** For accepted strings the simulator reconstructs one accepting computation path, not only the set trace.
- **Content-addressed automata.** The automaton ID is the hash of its canonical form, so clients cannot tamper with an automaton and identical requests are cached.
- **Closed grammar, honest refusals.** Anything outside the IR or outside regular languages is refused with a reason and, where possible, a regular alternative.

1.2 Goals and non-goals

Goals	Non-goals (v1)
Accept new descriptions dynamically within the supported scope	Understanding arbitrary prose, mathematics notation or other languages
Ask the fewest clarifying questions needed	Pushdown automata, Turing machines, Mealy/Moore machines
Produce table, diagram and trace from one model	Hand-drawn automaton editing (planned for v2)
Prove correctness to a stated, visible level	Claiming universal correctness for all strings without exact equivalence
Run on a laptop with no GPU; LLM optional	Storing user accounts or personal data

2. Design principles

#	Principle	Consequence in the design
P1	Single source of truth	The canonical automaton model is immutable. Table, diagram, validator and simulator read it; none stores its own copy of δ.
P2	Interpretation is untrusted	Rules or an LLM may propose an IR. Only schema-valid, size-limited, user-confirmed IR reaches the compiler.
P3	Compile deterministically	Same IR and alphabet always give the same automaton, state names and IDs.
P4	Prove, do not assert	No automaton is shown as “correct” without a recorded verification level.
P5	Refuse rather than fabricate	Non-regular, unsupported or oversized requests return an error code and explanation.
P6	Disclose every assumption	Defaults (alphabet, zero-count parity, overlap) appear in an editable assumptions panel.
P7	Small questions only	At most two clarification questions per request; each offers tappable choices.

3. System architecture

The system is a staged pipeline. Interpretation is the only stage that may involve a language model; everything after IR validation is deterministic algorithmic code.

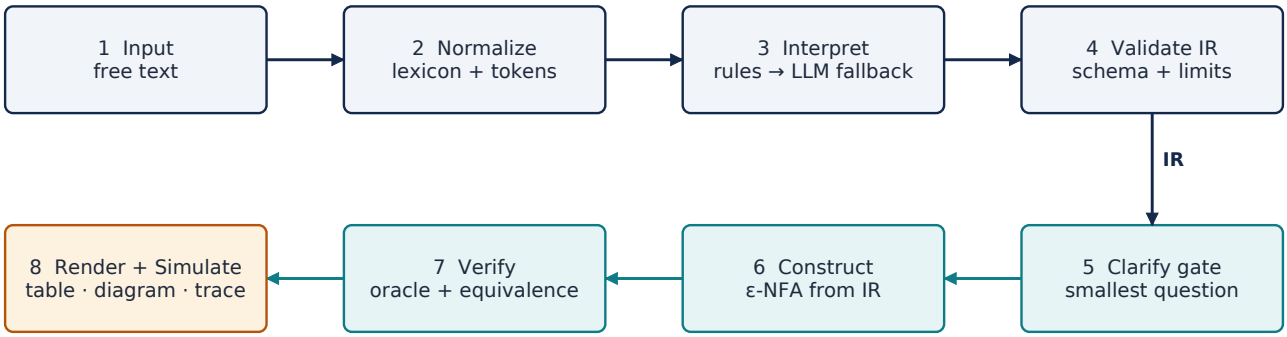


Figure 1 — Processing pipeline. Teal stages are deterministic; the clarify gate loops back to stage 3 with the user's answer merged into the partial IR.

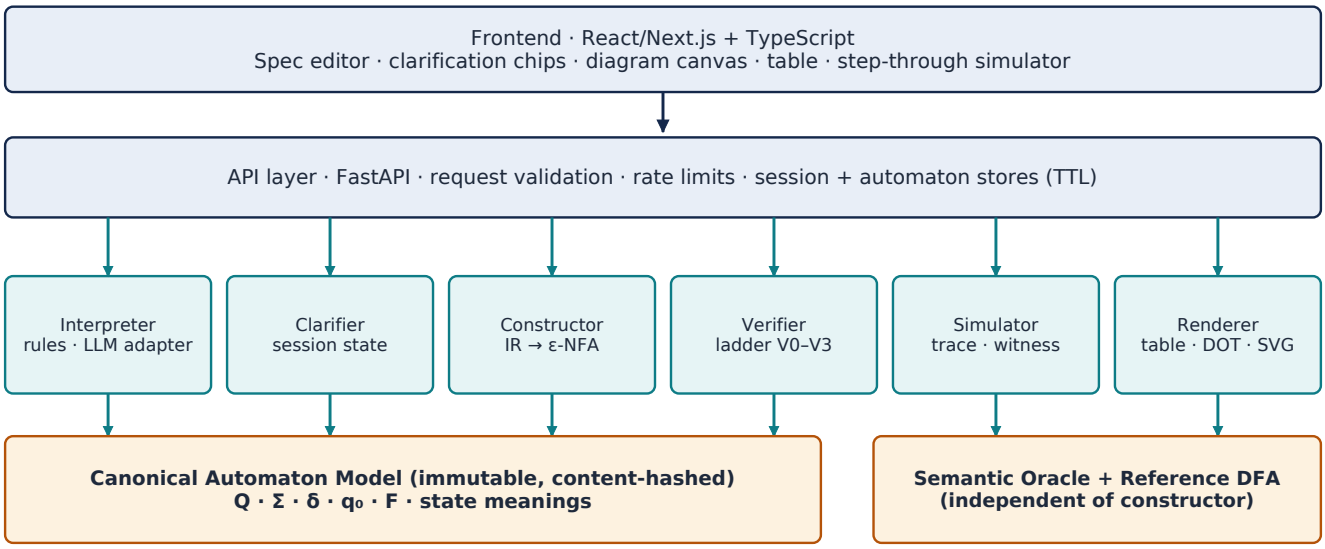


Figure 2 — Component architecture. The automata core never imports the LLM adapter.

3.1 Recommended stack

Layer	Choice	Reason
Backend	Python 3.11+, FastAPI, Pydantic v2	Typed IR validation, easy automata code, OpenAPI docs for free
Automata core	Pure Python package, no web imports	Testable in isolation; reusable in a CLI or notebook
Diagram	Graphviz (DOT → SVG) on the server; Cytoscape.js optional on the client	Reliable layout; Cytoscape adds highlighting and animation
Frontend	Next.js + TypeScript	Typed API client generated from OpenAPI
Stores	In-memory dict with TTL (v1); Redis later	Sessions and automata are small and short-lived
Testing	pytest, hypothesis, a fixed benchmark file	Property tests for semantics; benchmark for acceptance

3.2 Module layout

```
nfa_studio/
  core/        model.py  closure.py  simulate.py  determinize.py  product.py  minimize.py
  ir/          schema.py  normalize.py  limits.py  oracle.py
  compile/     atoms.py  regex_parser.py  thompson.py  combine.py  naming.py
  reference/   derivatives.py  to_dfa.py  equivalence.py
  verify/      ladder.py  counterexample.py
  interpret/   lexicon.py  rule_parser.py  ambiguity.py  nonregular.py  llm_adapter.py
  session/     store.py  clarify.py
  render/      table.py  dot.py  svg.py  json_graph.py
  api/         routes.py  errors.py  deps.py
tests/         unit/  property/  integration/  benchmark/benchmark_v1.json
```

4. Formal foundations used

The core model is an **ε-NFA** $N = (Q, \Sigma, \delta, q_0, F)$ with $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \to 2^Q$. An NFA without ε is the special case with no ε edges. The tool always stores the ε-NFA and can derive an ε-free view on demand.

Concept	Definition used in the implementation
ε-closure	ECLOSE(S) = all states reachable from S by zero or more ε edges. Computed by graph search with a visited set, so ε-cycles terminate. Precomputed per state and memoised.
Step	step(S, a) = ECLOSE($\bigcup_{q \in S} \delta(q, a)$)
Acceptance	Start with S_0 = ECLOSE({q_0}); after reading $w = a_1 \ldots a_n$, accept iff $S_n \cap F \neq \emptyset$. Passing through F earlier never causes acceptance.
ε-removal	$\delta'(q,a)$ = ECLOSE(δ(ECLOSE({q}), a)); q is accepting in the new machine iff ECLOSE({q}) ∩ F ≠ ∅.
Subset construction	DFA states are reachable subsets of Q (after ε-closure). Worst case $2^{\lvert Q \rvert}$ states; guarded by a state cap.
Product (AND)	For ε-free A and B: states (p,q), $\delta((p,q),a) = \delta_A(p,a) \times \delta_B(q,a)$, accepting iff both accept. Works directly on NFAs.
Union (OR)	New start state with ε edges to both starts; accepting set is the union.
Complement (NOT)	Determinize, complete with a dead state, then swap F and Q \F. Swapping F on an NFA is wrong and is forbidden in the code.
Language vs ε	The empty string ε is a string; the empty language ∅ contains no string. They are never conflated.

> **Decision:** ε-NFA is the canonical form. Union, concatenation and star use ε edges (Thompson style). Product and complement first apply ε-removal, and the construction explanation says so.

5. Language Specification IR (LSIR)

LSIR is a typed, closed tree. It is independent of the user's wording. A node type not listed here is rejected by schema validation, which makes the compiler total over everything it can receive.

5.1 Node catalogue

Node	Fields	Meaning
starts_with	pattern	w begins with pattern
ends_with	pattern	w ends with pattern
contains	pattern	w has pattern as a substring
avoids	pattern	w does not contain pattern (compiled as not(contains))
symbol_count	symbol, op ∈ {eq, ge, le}, n	number of occurrences of symbol compared with n
symbol_parity	symbol, parity ∈ {even, odd}	count of symbol is even or odd (zero is even)
length_mod	m, r	|w| mod m = r
pattern_count	pattern, op, n, overlap ∈ {true,false}	number of occurrences of pattern; overlap must be explicit
regex	expr	extended regular expression over the alphabet (concat, |, *, +, ?, grouping)
literal_set	words[]	w is exactly one of the listed finite words
and / or	children[]	intersection / union of child languages
not	child	complement relative to Σ*
concat / star	children[] / child	concatenation / Kleene star of child languages

5.2 Document shape

```
{
  "ir_version": "1.0",
  "alphabet": {"symbols": ["0","1"], "source": "user" | "inferred" | "default"},
  "language": {
    "type": "and",
    "children": [
      {"type": "symbol_parity", "symbol": "1", "parity": "even"},
      {"type": "contains", "pattern": "00"}
    ]
  },
  "assumptions": [
    {"id": "A1", "text": "Zero occurrences counts as even.", "editable": false}
  ],
  "clarifications": [],
  "limits": {"max_nodes": 40, "max_pattern_len": 16, "max_count": 20, "max_states": 200}
}
```

5.3 Normalisation and limits

- Flatten nested and/or, remove double negation, sort commutative children, and lower-case nothing: symbols are case-sensitive.
- Every symbol that appears in any pattern, regex or count must be in the alphabet; otherwise the IR is invalid.
- Hard limits: 40 IR nodes, patterns up to 16 symbols, counts up to 20, regex source up to 200 characters, alphabet up to 6 symbols. Limits live in one file and appear in `/v1/capabilities`.
- The normalised IR is serialised canonically (sorted keys, no whitespace). Its SHA-256 is the specification hash used for caching.

6. Interpretation and clarification

6.1 Two-tier interpreter

- **Tier 1 — rule parser.** A lexicon of phrases (“starts with”, “ends in”, “at least”, “even number of”, “divisible by”, “does not contain”, “either … or”, “both … and”) is matched on a tokenised, normalised sentence. Rules produce IR with a confidence score. High-confidence output skips the LLM.
- **Tier 2 — optional LLM adapter.** Used only when Tier 1 confidence is low. The model is asked for IR JSON conforming to the schema, at temperature 0, with the schema in the prompt. The reply is parsed as

data, never executed, and goes through the same validation as Tier 1.

- **Always user-confirmed.** The UI renders the IR as a readable sentence (“Strings over {0,1} that have an even number of 1s AND contain 00”) with editable chips before construction.

6.2 Ambiguity taxonomy

The clarifier asks a question only when the answer changes the recognised language. Otherwise it applies a documented default and lists it under assumptions.

Situation	Action	Default if not asked
“ending with ab”, no alphabet given	Show alphabet chip pre-filled with the pattern's symbols; ask only if wording hints at more symbols (“over letters”, “any characters”)	Symbols of the pattern, disclosed
“contains two a's”	Ask: exactly two or at least two?	None; must be answered
“contains aba twice”	Ask: may occurrences overlap?	None; must be answered
“even number of a's”	No question	Zero counts as even; disclosed
“no two consecutive a's”	No question	Interpreted as avoids(“aa”); disclosed
“starts and ends with the same symbol”	No question	Non-empty strings only; ε excluded; disclosed
“length divisible by 3”	No question	Includes length 0 (ε); disclosed
“a or b followed by c”	Ask: (a or b) then c, or a, or (b then c)?	None; must be answered
Conflicting constraints (empty language)	Warn after construction; still show the automaton	Empty language is valid output

6.3 Clarification state machine

- A session stores the original text, the partial IR, pending questions and the answers received. Answers are merged into the partial IR; interpretation is not restarted.
- At most two questions per request. If more are needed, the tool shows an editable IR form instead of more prompts.
- Each question has 2–4 mutually exclusive options plus “edit manually”. Sessions expire after 30 minutes.

6.4 Non-regular detection

Detection is rule-based on semantic patterns, with a refusal that names the reason and offers a regular alternative when one exists.

Pattern recognised	Why not regular	Offered alternative
Equal numbers of two symbols; $a^n b^n$; $a^n b^n c^n$	Needs unbounded counting	Bounded version: $n \le k$ (finite union); or $\lvert \#a - \#b \rvert \le k$
Palindromes; ww; reversals of unbounded strings	Needs unbounded memory	Palindromes up to length k
Prime length; perfect-square length; n!	Not an ultimately periodic set	Lengths in a stated finite set, or length mod m
“More a's than b's”	Needs unbounded counting	Difference bounded by k
Balanced parentheses of unbounded depth	Context-free, not regular	Depth ≤ k

Detection cannot be complete: some non-regular descriptions will not match a rule. That gap is closed by the compiler's own limits (state cap and timeout) and by verification, which will fail for any wrongly accepted request.

7. Construction library

Each construction records: preconditions, the meaning of every state, transitions, accepting states, a correctness argument and a size bound. The meaning strings are attached to states so the UI can explain them.

7.1 Atom constructions

Node	State meaning	Transitions	Accepting	Size
ends_with p (|p|=m)	q_i = guessed that the final match began and p[1..i] has been read since	q_0 loops on every symbol; $q_0 \to p_1 \to q_1 \to p_2 \to \ldots \to q_m$	q_m only; no outgoing edges, so the match must end the input	m+1
starts_with p	q_i = first i symbols equal p[1..i]	$q_i \to p_{i+1} \to q_{i+1}$; mismatch has no edge (branch dies); q_m loops on all Σ	q_m	m+1
contains p	As ends_with, plus a final loop on all Σ	q_0 loops on Σ; $q_0 \to \ldots \to q_m$; q_m loops on Σ	q_m	m+1
symbol_parity	e = even count, o = odd count	Counted symbol toggles e ↔ o; other symbols loop	e or o by request	2
symbol_count eq n	c_i = i occurrences seen (i ≤ n)	Counted symbol: $c_i \to c_{i+1}$; other symbols loop; extra occurrences have no edge	c_n	n+1
symbol_count ge n	c_i for i < n, then saturated c_n	c_n loops on Σ	c_n	n+1
symbol_count le n	c_i = i seen; overflow has no edge	Counted symbol: $c_i \to c_{i+1}$ up to c_n	All c_i	n+1
length_mod m,r	r_i = length mod m = i	every symbol: $r_i \to r_{(i+1) \bmod m}$	r_r	m
literal_set	trie node = prefix read	Trie edges	Word-end nodes	Σ|words|
regex	Thompson fragments	ε edges for | * ? ; concatenation by ε join	Fragment end	O(|expr|)

Why ends_with is genuinely nondeterministic: q_0 cannot know when the final occurrence of p begins, so it keeps scanning and also branches into the matching chain on every p_1. A branch reaches q_m and survives only if the input ends there, because q_m has no outgoing edge.

7.2 Combinators

Node	Method	Notes
or	New start with ε to each child start; accepting = union	Result stays an ε-NFA
concat	ε from each accepting state of A to start of B; accepting = F_B	Preserves nondeterminism
star	New start/accept state; ε loops back; ε to old start	Accepts ε by construction
and	ε-remove both, then product construction	No determinization required
not	ε-remove, determinize, complete, swap F	State cap and timeout apply; output becomes a DFA, labelled as such
avoids p	not(contains p), but using the KMP-style DFA for p directly when |p| is small	Avoids a subset blow-up; dead state explicit
pattern_count	Track occurrence count up to n+1 using a pattern-matching automaton plus a counter	overlap flag changes the failure transitions; both modes are tested separately

7.3 State-explosion policy

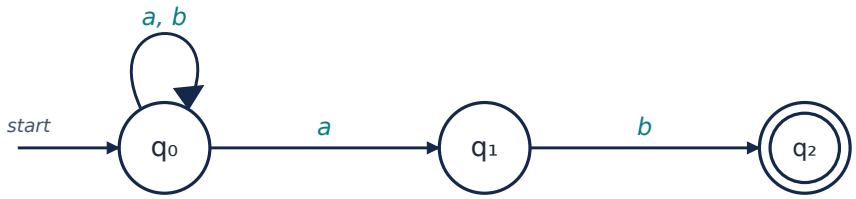
- Every operation that can grow exponentially (determinize, complement, product of large parts) runs under **max_states = 200** and a **2 second** wall-clock budget.
- On breach the API returns `E_STATE_LIMIT` with the partial size reached and a suggestion (simplify, shorten patterns, or split the request).

- Dead states and unreachable states are removed from the displayed diagram. A “show dead state” toggle restores it for educational use.
- The explanation text states when the displayed machine is a DFA produced by subset construction rather than a hand-built NFA.

7.4 Deterministic naming

States are renamed in breadth-first order from the start state, visiting symbols in alphabet order: q0, q1, q2, … . Equal IR with equal alphabet therefore always produces the identical automaton and identical hash. Display labels (for example the meaning string) are stored separately and never affect identity.

7.5 Worked construction: strings ending with ab over {a, b}



$\Sigma = \{a, b\}$ · $L = \{ x \cdot ab \mid x \in \Sigma^* \}$

State	a	b
→ q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	∅	$\{q_2\}$
* q_2	∅	∅

Meanings: q_0 = scanning (no commitment); q_1 = guessed that an “a” starts the final “ab”; q_2 = completed “ab” (accepts only if input ends).

Input	Active-set trace	Result
ab	$\{q_0\} \to \{q_0,q_1\} \to \{q_0,q_2\}$	Accept
aab	$\{q_0\} \to \{q_0,q_1\} \to \{q_0,q_1\} \to \{q_0,q_2\}$	Accept
bbab	$\{q_0\} \to \{q_0\} \to \{q_0\} \to \{q_0,q_1\} \to \{q_0,q_2\}$	Accept
aba	$\{q_0\} \to \{q_0,q_1\} \to \{q_0,q_2\} \to \{q_0,q_1\}$	Reject
abb	$\{q_0\} \to \{q_0,q_1\} \to \{q_0,q_2\} \to \{q_0\}$	Reject
a	$\{q_0\} \to \{q_0,q_1\}$	Reject
ε	$\{q_0\}$	Reject

These traces are regression fixtures only. The simulator computes traces from δ and never reads this table.

7.6 Worked construction: not containing aa over {a, b}

State	Meaning	a	b
→* s0	last symbol is not a (or start)	s1	s0
* s1	last symbol is a	dead	s0
dead	an aa has occurred	dead	dead

Both s0 and s1 accept. This is the complement of contains(aa): determinize contains(aa), complete it, swap F. The result is a 3-state DFA, which is also a valid NFA, and the explanation says it was produced by complement.

8. Verification ladder

Verification is the feature that turns the tool from “plausible” to “defensible”. It compares the compiled automaton against code that shares nothing with the compiler except the IR.

8.1 Two independent references

- **Semantic oracle** `matches(ir, w) → bool`. A direct recursive evaluator over the IR: for starts_with it calls `w.startswith(p)`; for symbol_parity it counts; for and/or/not it combines boolean results; for regex it uses a separate matcher. It builds no automaton.
- **Reference DFA** built by Brzozowski derivatives. Extended regular expressions (with intersection and complement) have derivatives defined directly, so every IR node except overlap-sensitive pattern_count translates to an extended regex. Derivatives are explored breadth-first with smart constructors and similarity rules to keep the set finite.

8.2 Levels

Level	Check	Evidence shown in UI
V0 Structural	States unique; δ sources/targets ∈ Q; labels ∈ Σ ∪ {ε}; q_0 ∈ Q; F ⊆ Q; table and diagram regenerate from δ with identical hash	“Well-formed”
V1 Bounded	Compare simulator vs oracle on every string up to length L (L chosen so total ≤ 200 000 strings; default 8–10)	“Verified on all strings up to length L”
V2 Exact	Determinize N, compare with the reference DFA by product BFS; any reachable state with differing acceptance gives the shortest counterexample	“Exactly equivalent to reference”
V3 Properties	Random strings, mutation tests, closure laws (De Morgan, double complement), ε-removal preserves language	“Property tests passed (n runs)”

- Badge colours: grey = V0 only; amber = V1; green = V2. Pattern-count with overlap reaches V1 and V3 only until a reference translation is added; the badge says so.
- A failed V1 or V2 never reaches the user as a diagram. The API returns `E_VERIFY_FAILED` with the counterexample and logs the IR for the developers.
- Verification results are cached with the automaton ID.

8.3 Counterexample search

```
def shortest_counterexample(dfa_n, dfa_ref):           # both complete over same alphabet
    start = (dfa_n.start, dfa_ref.start)
    queue, parent = deque([start]), {start: None}
    while queue:
        (p, q) = queue.popleft()
        if (p in dfa_n.final) != (q in dfa_ref.final):
            return rebuild_string(parent, (p, q))       # shortest by BFS order
        for a in alphabet:
            nxt = (dfa_n.delta[p][a], dfa_ref.delta[q][a])
            if nxt not in parent:
                parent[nxt] = ((p, q), a); queue.append(nxt)
    return None                                         # equivalent
```

9. Simulation engine

9.1 Algorithm

```
def simulate(auto, word):
    validate_symbols(word, auto.alphabet)               # E_SYMBOL_NOT_IN_ALPHABET
    before = {auto.start}
    after  = eclose(auto, before)
    steps  = [Step(index=0, symbol=None, before=before, after=after,
                   accepting=bool(after & auto.final))]
    for i, a in enumerate(word, start=1):
        moved = set().union(*(auto.delta.get((q, a), set()) for q in after)) if after else set()
        closed = eclose(auto, moved)
        steps.append(Step(i, a, moved, closed, bool(closed & auto.final)))
        after = closed
    accepted = bool(after & auto.final)
    witness = reconstruct_path(auto, word, steps) if accepted else None
    return Trace(word, steps, accepted, witness)
```

9.2 Trace record

Field	Meaning
index	Position in the input (0 for the initial set)
symbol	Symbol consumed at this step; null at index 0
moved	Set reached by δ before ε-closure
after	Active set after ε-closure
accepting_present	True if after ∩ F is non-empty (informational only)
final_accepted	Set only on the last step and equals the acceptance decision
witness	Optional list of (state, symbol, state) edges forming one accepting path, reconstructed with per-step predecessor pointers

9.3 Behaviour rules

- Empty input evaluates ECLOSE($\{q_0\}$) ∩ F. An empty active set stays empty; all later steps are no-ops and the diagram shows a “no active states” notice.
- A missing transition removes that branch. Multiple accepting states and multiple active states are handled by sets; no code assumes one active state.
- Limits: input length up to 10 000 symbols; the trace is paginated 200 steps per page; batch simulation up to 100 strings per call.
- Complexity: O(|w| · |Q| · d) per word, where d is the largest destination-set size. Closures are memoised once per automaton.

9.4 Interactive behaviour

- Step forward/back, play at adjustable speed, jump to step, and a scrubber over the whole input.
- The diagram highlights the current active states and the edges taken at the current step; accepted strings can also show the witness path in a different colour.
- A history list stores each tested string with its verdict for quick re-run against the same automaton.

10. Table and diagram generation

10.1 Transition table

- One column per symbol in alphabet order, then an ε column only if the automaton has ε edges. Each cell is the sorted destination set, or ∅.
- The start state is marked → and accepting states *; a state that is both is marked →*.
- Export as CSV, Markdown, LaTeX tabular and JSON. A “show ε-free version” toggle produces the table of the ε-removed machine.

10.2 State diagram

- Generated from the same model into Graphviz DOT, then SVG. A JSON graph (nodes, edges, labels) feeds the client-side highlighter.
- Parallel edges between the same pair of states merge into one label (“a, b”); ε edges are dashed and labelled ε.
- Style: left-to-right layout, accepting states as double circles, an invisible node supplies the start arrow, dead states hidden by default.
- Labels on large machines can show only q-numbers with meanings in a legend panel to avoid clutter.
- Export SVG, PNG and DOT.

10.3 Consistency check

A function `model_hash(auto)` returns the SHA-256 of the canonical JSON of (Q, Σ, δ, q_0, F). The table renderer and diagram renderer each return the hash of the model they read. The API refuses to respond if either hash differs from the automaton ID, which makes divergent hard-coded copies impossible.

11. API contract (v1)

Endpoint	Purpose
POST /v1/interpret	Text in; IR or a clarification question out; creates a session
POST /v1/interpret/{session}/answer	Merge a clarification answer into the partial IR
POST /v1/automata	Construct and verify from a confirmed IR; returns automaton_id
GET /v1/automata/{id}	Five-tuple, δ, explanation, assumptions, verification report
GET /v1/automata/{id}/table	Table data in json, csv, md or tex
GET /v1/automata/{id}/diagram	Diagram as svg, dot or json graph
POST /v1/automata/{id}/simulate	One word; full trace with optional witness
POST /v1/automata/{id}/simulate/batch	Up to 100 words against one automaton
POST /v1/automata/{id}/verify	Re-run or deepen verification (choose level)
GET /v1/capabilities	Supported nodes, limits, versions
GET /v1/health	Liveness and readiness

Simulation and verification take the automaton **by ID only**. The server never accepts a client-supplied δ for these calls, which resolves the trust problem in earlier designs.

11.1 Examples

```
POST /v1/interpret
{ "text": "strings over {a,b} that contain aba twice", "session_id": null }

200 OK
{ "status": "clarification_required",
  "session_id": "s_4f9c",
  "question": { "id": "Q1", "text": "May the two occurrences of aba overlap (as in ababa)?",
                "options": [ {"id":"yes","label":"Yes, overlap allowed"},
                             {"id":"no","label":"No, they must be disjoint"} ] },
  "partial_ir": { "alphabet": {"symbols":["a","b"],"source":"user"},
                  "language": {"type":"pattern_count","pattern":"aba","op":"ge","n":2} } }

POST /v1/automata/{id}/simulate        { "word": "bbab", "witness": true }

200 OK
{ "automaton_id": "sha256:9b1e…", "word": "bbab", "valid": true,
  "steps": [ {"index":0,"symbol":null,"after":["q0"]},
             {"index":1,"symbol":"b","moved":["q0"],"after":["q0"]},
             {"index":2,"symbol":"b","moved":["q0"],"after":["q0"]},
             {"index":3,"symbol":"a","moved":["q0","q1"],"after":["q0","q1"]},
             {"index":4,"symbol":"b","moved":["q0","q2"],"after":["q0","q2"],"accepting_present":true} ],
  "accepted": true,
  "witness": [["q0","b","q0"],["q0","b","q0"],["q0","a","q1"],["q1","b","q2"]] }
```

11.2 Error codes

Code	HTTP	When
E_SCHEMA_INVALID	422	IR fails schema or normalisation
E_ALPHABET_MISSING	422	No alphabet and none safely inferable
E_NEEDS_CLARIFICATION	200	A question is pending (status field, not an error page)
E_UNSUPPORTED_NONREGULAR	422	Request recognised as non-regular; alternative offered when known
E_UNSUPPORTED_FEATURE	422	Regular but outside LSIR node set
E_SYMBOL_NOT_IN_ALPHABET	422	Simulation word contains a symbol outside Σ; position reported
E_INPUT_TOO_LONG	413	Word exceeds the length limit
E_STATE_LIMIT	422	Construction exceeded max_states
E_TIMEOUT	504	Construction exceeded its time budget
E_VERIFY_FAILED	500	Automaton disagrees with reference; counterexample included
E_AUTOMATON_NOT_FOUND	404	Unknown or expired automaton ID
E_RATE_LIMITED	429	Too many requests

12. User experience

12.1 Main flow

Step	Screen	What the user sees
1	Describe	Text box, example chips, alphabet field with the inferred value shown as an editable chip
2	Confirm	Plain-English reading of the IR, assumptions with toggles, and any one or two clarification questions as buttons
3	Build	Progress strip: compiled → verified; verification badge with level and L
4	Explore	Tabs: Diagram · Table · Formal definition · Explanation; ε-free toggle
5	Simulate	Input box, step controls, active-state highlighting, history, witness path

12.2 Quality requirements

- Every error message names the reason and offers the next action (for example “Use at most 16 symbols per pattern” or “Try $a^n b^n$ with $n \leq 4$”).
- Keyboard operable; diagram nodes have text alternatives; colours never carry meaning alone (accepting = double circle, active = bold outline plus fill).
- Typography: bundle a font with full Greek and mathematical symbol coverage so σ, ε, δ, Σ, ∅ and subscripts render everywhere; test exports for glyph loss.

13. Security and model governance

Risk	Control
Prompt injection through user text	LLM output treated as data; strict schema parse; no tool access; node, pattern and regex limits; user must confirm the IR
Credential exposure	AI-provider keys only on the backend, read from environment; never sent to the browser; redacted in logs
Regex denial of service	Own regex parser to NFA (no backtracking engine); size limit; time budget
State explosion	State cap, wall-clock budget, E_STATE_LIMIT
Client tampering	Automata referenced by content hash; simulation by ID only
Resource abuse	Per-IP rate limits; input length and batch limits; TTL on stores
Wrong answers presented as true	Mandatory verification; failed automata never rendered
LLM drift	Temperature 0; pinned model id; interpretation cache by normalised text; benchmark rerun on every model change

14. Testing and acceptance

14.1 Test layers

Layer	Examples
Unit	ε-closure with cycles; step on empty set; each atom on hand-computed strings; naming determinism; table and diagram regeneration
Property (hypothesis)	For random IR trees and random strings: simulator(compiled) = oracle; double complement; De Morgan; ε-removal preserves language; union/intersection laws
Equivalence	Determinized compiled machine vs derivative-based reference DFA for every benchmark entry
Integration	interpret → clarify → construct → verify → render → simulate, with and without the LLM adapter
Negative	Out-of-alphabet symbols, oversized inputs, non-regular requests, schema violations, expired IDs
Rendering	Golden SVG/DOT for small machines; glyph check on exported PDFs and PNGs

14.2 Benchmark of unseen prompts (benchmark_v1)

A fixed file of 40 prompts with expected IR (or expected clarification or refusal). A sample is shown; the file is extended to 40 before Phase 8. Prompts are written independently of the implementation and are not used during development tuning.

#	Prompt	Expected outcome
1	binary strings ending in 01	ends_with “01”, alphabet {0,1}
2	strings over {a,b,c} starting with ca	starts_with “ca”
3	words containing the substring abba over {a,b}	contains “abba”
4	even number of 1s	symbol_parity; zero-is-even assumption listed
5	exactly three b's over {a,b}	symbol_count eq 3
6	at most two a's	symbol_count le 2; alphabet question or chip
7	strings that contain two a's	Clarification: exactly or at least
8	contains aba twice	Clarification: overlap
9	no two consecutive 1s	avoids “11”
10	length divisible by 3 over {0,1}	length_mod 3,0
11	starts with a AND ends with b	and(starts_with, ends_with)
12	contains 00 OR ends with 11	or(contains, ends_with)
13	even number of a's but not containing bb	and(parity, not(contains))

#	Prompt	Expected outcome
14	(ab)* over {a,b}	regex
15	strings equal to one of: 0, 10, 110	literal_set
16	same number of a's and b's	Refuse: non-regular; offer difference ≤ k
17	$a^n b^n$ for all n	Refuse: non-regular; offer n ≤ 4
18	palindromes over {0,1}	Refuse: non-regular; offer length ≤ k
19	strings whose length is prime	Refuse: non-regular
20	strings that are good	Refuse or ask: not interpretable

14.3 Acceptance criteria

Criterion	Threshold
Benchmark outcomes correct (IR, clarification, or refusal)	≥ 90% rule-only; ≥ 95% with LLM tier
Compiled automata that reach V2 (exact) or V1 (bounded, overlap-count only)	100% of those shown to users
Simulation agrees with oracle on all strings up to length 10 for benchmark entries	100%
Table and diagram hash equals automaton ID	100%
Median response time for construct + verify on benchmark	< 1 second without LLM
Tests reported as passing	Only after the suite has actually been executed in CI, with the report attached

15. Implementation roadmap

Phase	Weeks	Deliverables	Exit criteria
0 Foundations	1	Repo, CI, OpenAPI skeleton, benchmark file stub, glyph-safe fonts	CI runs an empty test suite green
1 Core model	2–3	Model, closure, simulator, determinize, product, complement; IR schema; oracle	Simulator = oracle on hand-built machines; V0 checks pass
2 Constructions	3–5	All atoms and combinators, Thompson regex parser, naming, state meanings	Every node type has unit and property tests
3 Verification	5–6	Derivative reference compiler, equivalence, counterexamples, ladder, badges	Injected bug in a construction is caught with shortest counterexample
4 Rendering	6–7	Table exporters, DOT/SVG, JSON graph, hash consistency	Golden outputs stable; hash check enforced
5 Interpretation	7–9	Lexicon, rule parser, ambiguity taxonomy, clarifier, non-regular rules	≥ 90% on benchmark without LLM
6 LLM adapter (optional)	9–10	Schema-constrained prompts, validation, cache, injection tests	≥ 95% on benchmark; no unvalidated IR path
7 Frontend	10–12	Describe/Confirm/Explore/Simulate screens, highlighter, step-through	End-to-end demo of five unseen prompts
8 Hardening	12–13	Rate limits, error-message audit, docs, accessibility pass, final benchmark	All acceptance criteria in 14.3 met with a recorded run

16. Risks and open decisions

Risk / decision	Mitigation or recommendation
Rule parser coverage is narrow at first	Start with the 20 benchmark patterns; grow the lexicon from logged unmatched phrases; LLM tier is the fallback
Derivative reference can grow large	Smart constructors and similarity rules; fall back to V1 with an honest badge
Complement/product blow-ups hurt usability	State cap, explanation of why, and a suggestion to simplify
Students expect “textbook” NFAs, not DFAs	Prefer direct NFA constructions; say clearly when the result came from determinization
Open: minimise displayed machines?	Offer an optional “minimal DFA” view later; v1 shows the construction as built (minus dead states)
Open: Unicode symbols in alphabet	v1: single printable characters only; multi-character tokens deferred
Open: authentication	Not needed for v1; add if hosted publicly

Appendix A. Glossary

Term	Meaning
Alphabet Σ	Finite non-empty set of symbols
ε	The empty string (length 0); also the label of a spontaneous transition in an ε-NFA
Active set	Set of states in which at least one computation path currently sits
Witness path	One concrete accepting sequence of transitions for an accepted string
LSIR	Language Specification IR: the typed tree between interpretation and compilation
Oracle	Direct evaluator of the IR on a string; builds no automaton
Reference DFA	Automaton built by derivatives from the IR, independent of the production compiler
Verification level	V0 structural, V1 bounded, V2 exact, V3 properties
Canonical model	The one immutable automaton from which all outputs are derived

Appendix B. Pseudocode: ε-closure and reconstruction

```
def eclose(auto, S):
    seen, stack = set(S), list(S)
    while stack:
        q = stack.pop()
        for r in auto.delta.get((q, EPS), ()):
            if r not in seen:
                seen.add(r); stack.append(r)
    return seen                                  # terminates on ε-cycles

def reconstruct_path(auto, word, steps):
    # steps[i].pred maps each state in steps[i].after to (prev_state, symbol_or_EPS)
    q = next(iter(steps[-1].after & auto.final))
    path = []
    for i in range(len(word), 0, -1):
        prev, a = steps[i].pred[q]
        path.append((prev, a, q)); q = prev
    return list(reversed(path))
```

End of specification.